Northern Azure User Group

I don't really understand Azure DNS…

…and at this point I am too embarrassed to ask.

scan me · linktr.ee/simonpainter

Simon Painter — Multi Cloud Network Architect, Microsoft MVP, and AWS Certification SME. I spent the first few years of my career putting Active Directory DNS *into* enterprises, and the last few helping pull it back *out* into Azure. This is the talk I wish someone had given me.

AZ-700 | AWS Advanced Networking | AWS Solutions Architect | OCI Architect | Terraform Associate | Aviatrix ACE

The route

Where we're going

1. Authoritative servers vs resolvers
2. How Active Directory blurred the lines
3. The Magic IP & the DHCP set
4. Azure DNS Private Resolver
5. Reference architectures
6. Private Link DNS (recap)
7. Azure Firewall as DNS proxy
8. A detour: running an API over DNS
9. DNS security policy
- ✓ Wrap-up & questions

If you only remember one slide, make it the next one.

Foundations

Two jobs people smush into one word

Authoritative server vs recursive resolver

Most “DNS is broken” tickets are one of these two roles pointed at the wrong thing.

Foundations

Forwarding ≠ recursing

Conditional forwarding is just “forward, but only for this domain”

Your resolver

can hand queries to another resolver instead of recursing

→ **Forwarder**

send **everything** to this upstream

→ **Conditional forwarder**

corp.local → on-prem DNS · everything else → recurse normally

This is the everyday hybrid-DNS lever — and exactly what Azure's forwarding rulesets are.

…and why your brain resists

Active Directory blurred the lines

AD-integrated DNS: one box, three jobs

Authoritative

for your AD domain

Recursive

for everything else

Forwarder

conditional rules

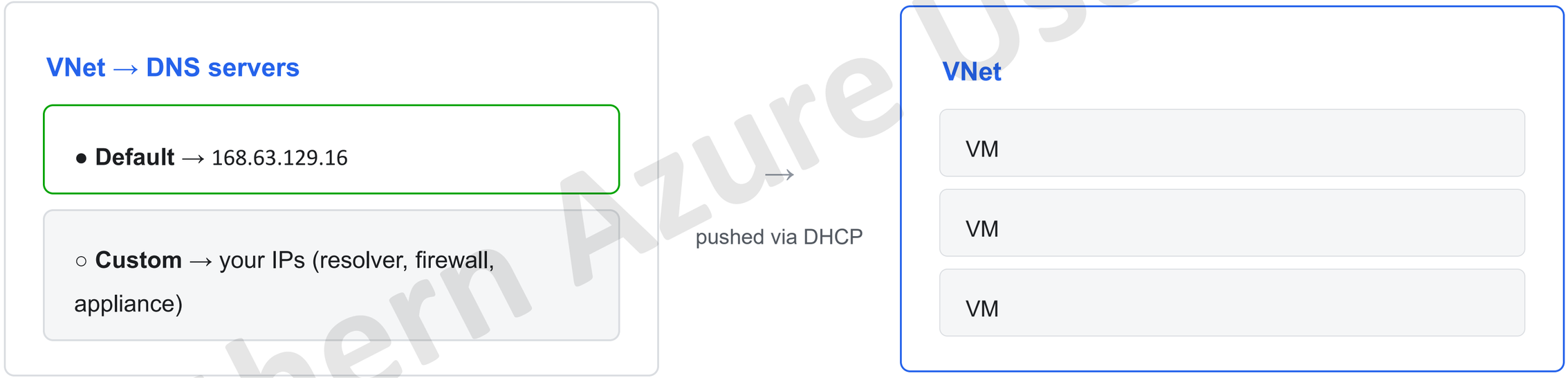
Bonus: this coupling is why DNS often lives with the **identity team, not networks** .

Zones tied to domain-controller placement → hybrid means DCs-in-cloud or a WAN dependency. Azure makes you un-smush the roles.

Azure-provided DNS

The VNet “DNS servers” toggle

It's just a DHCP option — which is why nothing changes until a reboot



VMs keep their old DNS until they renew the lease or reboot — change the setting and nothing happens.

Using a VPN / ExpressRoute gateway? Keep `168.63.129.16` in the list.

The plumbing every VNet already has

168.63.129.16 — the Magic IP

Subnet 10.0.0.0/24 reserved IPs

.0	network
.1	gateway
.2 / .3	**“Azure DNS” — doesn't answer!**
.255	broadcast

The real answers come from the magic IP — a VIP of the host node, static in every region, not subject to UDRs.

One IP, many hats

- DNS resolver (default via DHCP)
- DHCP server identity
- VM agent endpoint :32526
- Load-balancer health-probe source

The DHCP lease itself pushes a /32 static route to the magic IP via the gateway — a UDR can't black-hole it.

```
$ sudo dhcpcd --dumplease eth0
classless_static_routes=0.0.0.0/0 10.0.0.1
168.63.129.16/32 10.0.0.1 169.254.169.254/32 10.0.0.1

domain_name_servers=168.63.129.16
```

Security gotcha

Your NSG does not block the Magic IP

```
$ dig example.com +short @8.8.8.8 ;; error  timed out      ← NSG blocks

$ dig example.com +short
@168.63.129.16   104.21.53.33 ← still works!
```

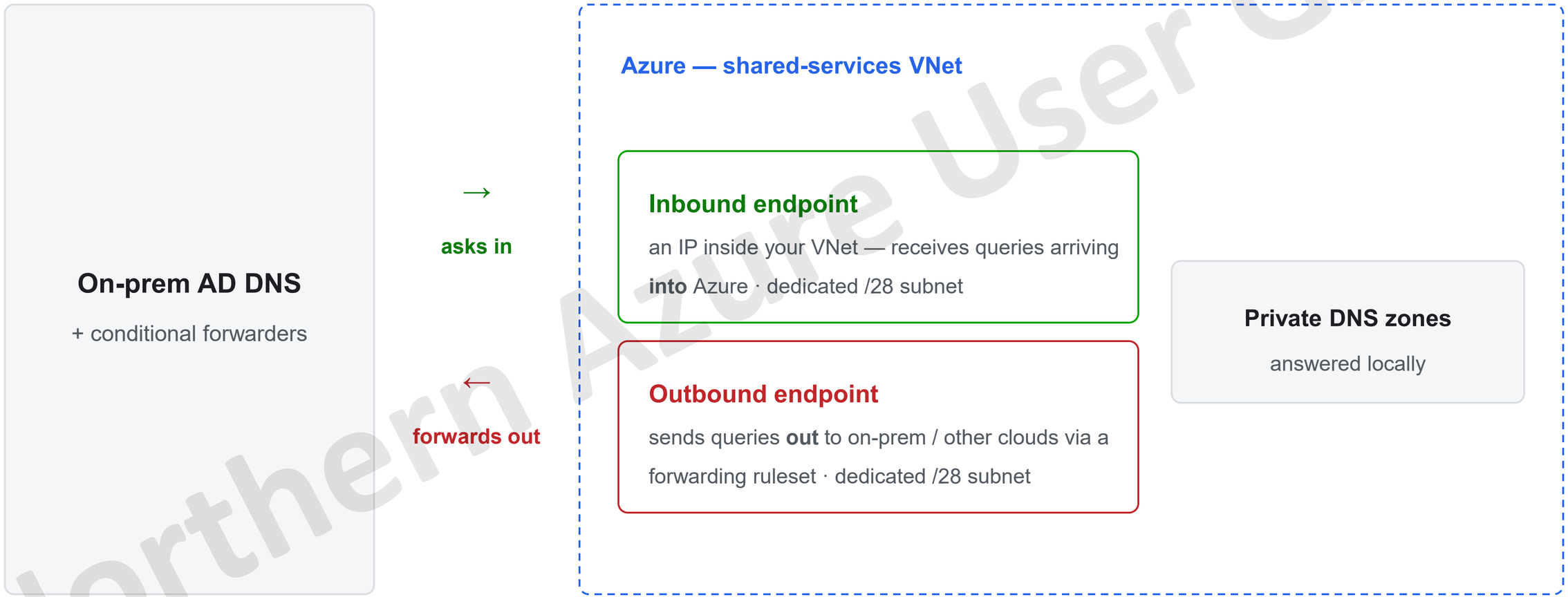
A deny rule on port 53 to **any** destination still lets DNS reach 168.63.129.16 — so DNS exfiltration works works even when you've “forced” a custom appliance.

Fix: deny with the **AzurePlatformDNS** service tag, raw port-53 rule.

Same family as AzureLoadBalancer and AzurePlatformIMDS.

The managed resolver

Azure DNS Private Resolver: in & out



Managed HA, no VMs to patch. Inbound = ingress · outbound = egress. Subnets delegated to Microsoft.Network/dnsResolvers.

The managed resolver

Forwarding rulesets = conditional forwarding, managed

DNS forwarding ruleset (up to 1,000 rules)

corp.local	→ 10.1.0.10
onprem.com	→ 10.1.0.10
azure.contoso.com	→ inbound endpoint
“.” (default)	→ **leave to Azure ✓**

→

linked to

- **VNet 1**
- **VNet 2**
- **VNet 3**

one ruleset, many VNets — no peering needed

And the reverse: one outbound endpoint supports **two rulesets** — different VNets get different rules, but share the same egress.

Don't accidentally forward “.” to on-prem — many PaaS services need public DNS to keep working.

The one slide to remember

Magic IP vs Private Resolver

Magic IP

✓ default in every VNet

✓ reachable from any VNet

✗ not reachable from on-prem

Private Resolver

✓ inbound answers on-prem queries

✓ outbound forwards to on-prem

✓ managed, HA, scalable

✓ rulesets, per-VNet links

~ costs money + needs /28 subnets

“An **inbound endpoint** answers on-prem queries. The **magic IP** cannot. That's the whole reason it exists.”

Reference architecture

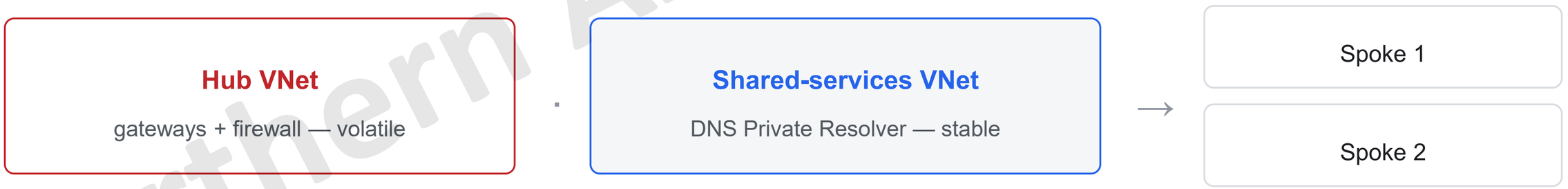
Two principles first

1 · DNS-first

Teams plan ExpressRoute / VPN / SD-WAN first — but great connectivity is useless if apps can't find each other.

2 · Resolver in shared services, not the hub

Keep connectivity and DNS in separate blast radii — a firewall change shouldn't take name resolution down with it.

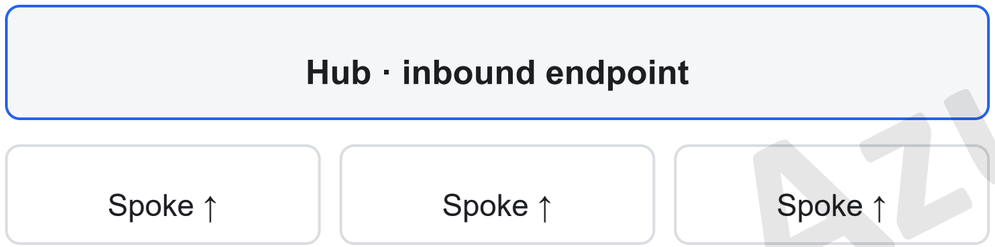


Reference architecture

Centralized vs distributed

Centralized — custom DNS

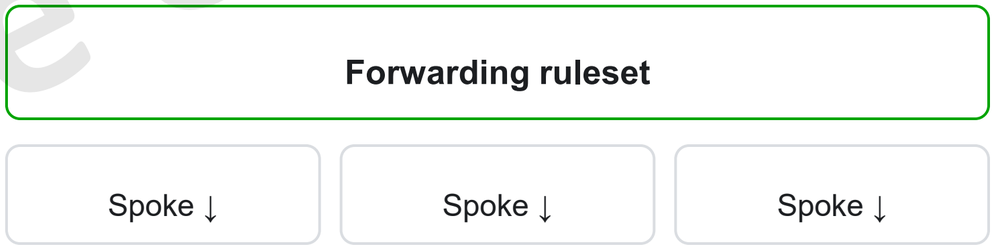
Spokes point their VNet DNS at the hub inbound endpoint. All DNS funnels through the hub; zones link only to the hub.



One chokepoint · control · logging · easy to put a firewall in path

Distributed — ruleset links

Spokes keep Azure-provided DNS and get a ruleset link. Resolution stays local; rules applied per-VNet.



No single chokepoint · resilience · scales without touching spokes

Reference architecture

Multi-region: global zone, regional resolvers

One global private DNS zone

zones are global resources — a single zone survives a regional outage

Region A

Private Resolver

Region B

Private Resolver

On-prem DNS

every resolver forwards to all on-prem servers

One resolver per region · zones global, resolvers regional · a regional outage never blocks on-prem lookups.

Private Link & DNS · covered in detail last session

Private endpoint DNS in three lines

1. Once a private endpoint exists, the public FQDN is a CNAME to privatelink.* — **the zone link decides the answer** : no zone → public IP over the internet; zone linked → private IP.

2. The **DNS zone group** binds endpoint ↔ zone, so A records are created, updated and deleted with the endpoint's lifecycle — no manual record-keeping.

3. At scale: **one central set of privatelink.* zones** + Azure Policy DeployIfNotExists auto-wires the zone group — app teams can't get it wrong.

What matters for today: those zones are **authoritative** — the resolver path we're building has to be linked to them.

Inspection in the path

Why DNS has to go through the firewall

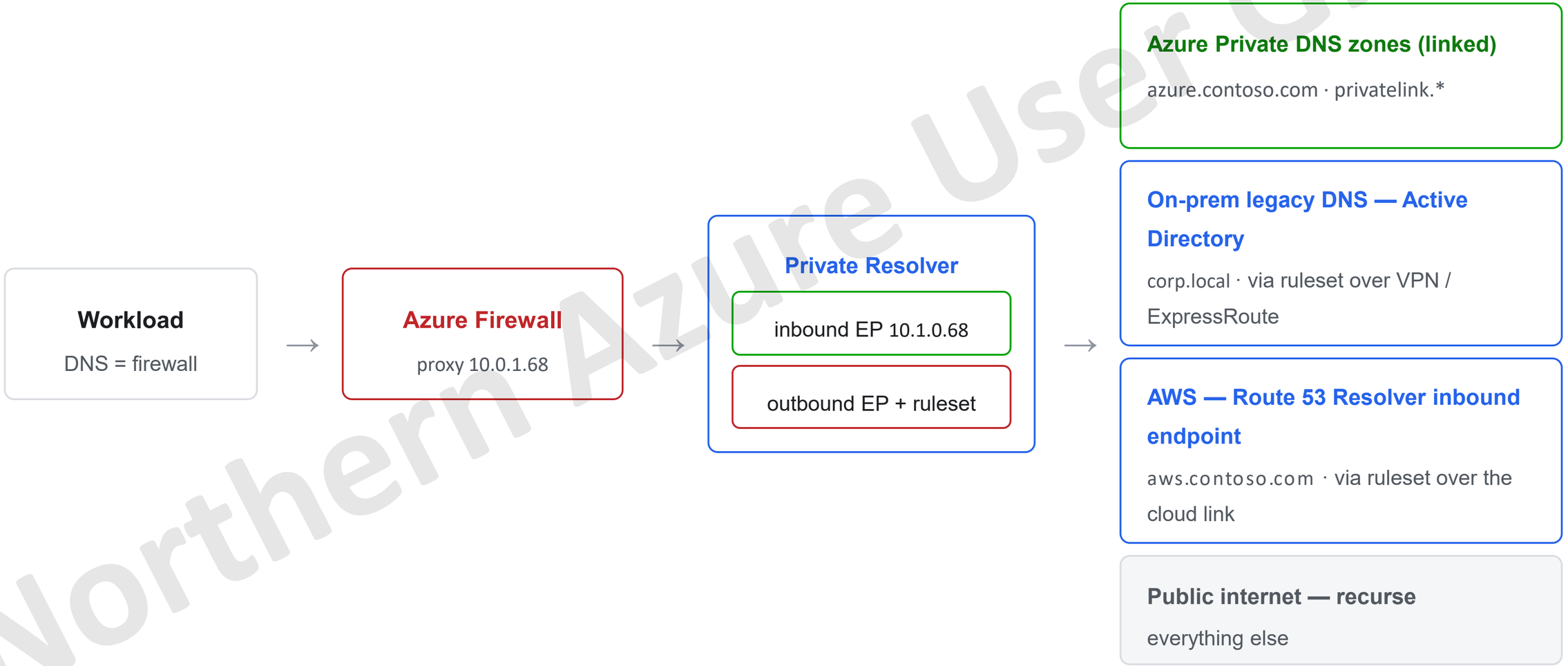
FQDN-based **network rules** only work if the firewall and the clients resolve names **identically** . Resolve elsewhere and the client may get a different IP than the firewall cached → **rule misses, leaky or broken traffic**.

Bonus: DNS query logging for the whole VNet, and FQDN filtering — e.g. lock NTP to `time.windows.com`.

Inspection in the path

Inbound & outbound: the hybrid resolution path

firewall → resolver → private zones · on-prem legacy DNS · AWS · public



The proxy **forwards** , it doesn't merge zones — the upstream must see them. Answers are non-authoritative; if on-prem needs authoritative replies, run real authoritative servers on Azure VMs.

Aside · because DNS is more flexible than you'd like

You can run an API over DNS

A Pokémon type lookup served entirely as DNS TXT records

client / dig
any resolver

⇄

DNS API proxy
a tiny Azure VM

⇄

PokeAPI
pokeapi.co

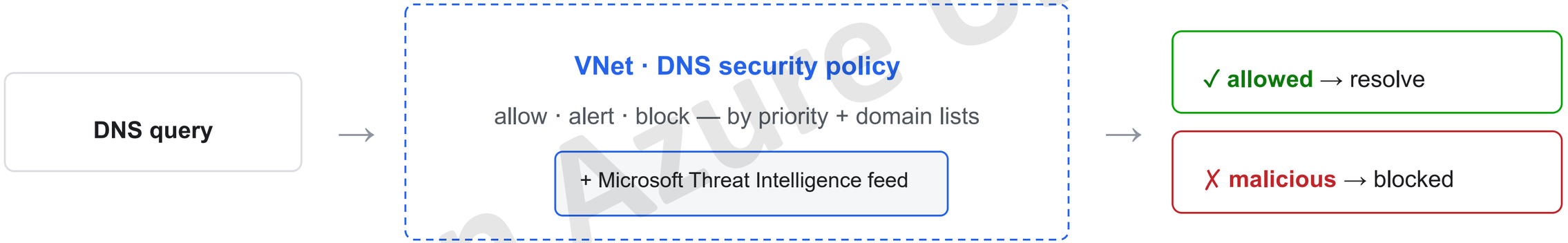
```
$ dig TXT mewtwo.pokemon.api2dns.simonpainter.com +short
"psychic"
$ dig TXT pikachu.pokemon.api2dns.simonpainter.com +short
"electric"
```

the write-up

The serious bit: DNS is the quiet protocol nobody watches. TXT records can carry **arbitrary data** — handy for lookups, perfect for **exfiltration & C2**. Not logging DNS = a visibility gap.

Modern add-on

DNS security policy: filter + log at the VNet



Logs every query — public and private — to **Log Analytics, storage or Event Hubs** .

One policy per VNet, same region. Complements zones, resolver and firewall — doesn't replace them.

Current developments · a very old protocol, still evolving

Encrypted DNS finds its own way

Encrypted transports — DoH / DoT / DoQ

Plaintext port 53 is on the way out. Windows DNS Server now has DNS-over-HTTPS in public preview — encrypted DNS is arriving in the enterprise stack, not just browsers.

New record types — SVCB / HTTPS (RFC 9460)

Records that carry **service metadata** , not just addresses: protocols, ports, endpoints, preferences. CNAME-at-apex solved; HTTP/3 advertised up front.

DDR — resolvers advertise their encrypted selves

Clients ask their plaintext resolver one bootstrap question, then silently upgrade to DoH/DoT:

```
dig SVCB _dns.resolver.arpa @1.1.1.1
```

Why you should care

Clients can auto-discover **external** encrypted resolvers and quietly bypass the inspection path we just built. Mitigation: block SVCB/HTTPS record types at your resolver — and keep watching port 443.

Current developments · a very old protocol, still evolving

DNS-AID: AI agent discovery over DNS

Today: hand-wired agents

hardcoded endpoints · metadata in docs · trust out of band

→

DNS-AID: publish discovery in DNS

SVCB records under domains you already own — chatbot.example.com

Why DNS is the right substrate

Delegation & ownership tied to domains · public and private namespaces · caching + TTLs · the ops tooling already exists. Same spirit as SRV, DNS-SD, OIDC discovery.

And it maps onto today's talk

Split-horizon & private zones for internal-only agents, DNSSEC for origin trust — the authoritative-vs-resolver model carries straight over. IETF draft + Linux Foundation backing; reference stack exists.

45-year-old protocol, brand-new workload — DNS keeps being the discovery layer everything else stands on.

Putting it together

It was two ideas all along

Everything we covered is one of these two roles wearing Azure clothing

Authoritative

“owns the records”

- Public DNS zones
- Private DNS zones
- privatelink.* zones

Resolver

“finds the answer”

- Magic IP (basic)
- Private Resolver (in / out + rulesets)
- + firewall proxy in the path
- + DNS security policy

Take it home

Cheat sheet & questions

Custom DNS vs ruleset	chokepoint + inspection vs local + scale
Firewall in the DNS path?	only for FQDN network rules or full DNS logging
Private zones	link where clients resolve; let Policy manage records
On-prem ↔ Azure ↔ Other Cloud	Legacy & multi cloud resolution with inbound and outbound resolver endpoints

Thanks — questions?

write-ups at simonpainter.com

all the DNS posts →